

Una entidad nacional de supervisión logística quiere desarrollar una aplicación para controlar el rendimiento de distintos **centros operativos**.

Cada centro trabaja de **lunes a viernes** y en **dos turnos diarios**:

- mañana
- tarde

De cada centro se quiere almacenar semanalmente dos tipos de información:

- una **matriz `int[5][2]` de operaciones completadas**
- una **matriz `int[5][2]` de incidencias registradas**

Es decir, para cada día y turno se conoce:

- cuántas operaciones se han completado,
- cuántas incidencias se han producido.

Además, cada centro tiene asignado un **responsable técnico**.

La aplicación debe distinguir dos tipos de centros:

- **CentroDistribucion**
- **CentroSeguridad**

Todos los centros comparten información común, pero su índice de eficiencia se calcula de forma distinta.

Por ello, el programa debe usar **clase abstracta, interfaz, enum, herencia, sobrescritura, polimorfismo, matrices y arrays de objetos**.

SE PIDE

1) Enumerado

Crea un enum llamado **Zona** con estos valores:

- NORTE
 - SUR
 - ESTE
 - OESTE
 - CENTRAL
-

2) Clase auxiliar

Crea una clase llamada `Responsable` con:

Atributos

- `String dni`
- `String nombre`
- `int antiguedad`

Métodos

- constructor
 - getters
 - `toString()`
-

3) Interfaz

Crea una interfaz llamada `Supervisable` con el método:

`boolean necesitaAuditoria();`

Este método indicará si el centro debe pasar una auditoría extraordinaria.

4) Clase abstracta

Crea una clase abstracta llamada `CentroOperativo` que implemente `Supervisable`.

Atributos

- `String codigo`
- `Zona zona`
- `Responsable responsable`
- `int[][] operaciones` → matriz 5x2
- `int[][] incidencias` → matriz 5x2

Métodos obligatorios

- constructor con parámetros
- getters
- `toString()`

- `int calcularTotalOperaciones()`
- `int calcularTotalIncidencias()`
- `int calcularOperacionesDia(int dia)`
- `int calcularIncidenciasDia(int dia)`
- `double calcularTasaIncidencias()`

La tasa de incidencias se calcula así:

$(\text{totalIncidencias} * 100.0) / \text{totalOperaciones}$

Si el total de operaciones es 0, la tasa será 0.

- `void mostrarResumenSemanal()`

Debe mostrar, para cada día:

- operaciones mañana
- operaciones tarde
- incidencias mañana
- incidencias tarde
- método abstracto:

`double calcularIndiceEficiencia();`

5) Subclases

a) `CentroDistribucion`

Hereda de `CentroOperativo`.

Atributo propio

- `int paquetesUrgentes`

Índice de eficiencia

Se calcula así:

$\text{indice} = \text{calcularTotalOperaciones()} - \text{calcularTotalIncidencias()} + \text{paquetesUrgentes} * 1.5$

Auditoría

Un centro de distribución necesita auditoría si se cumple al menos una de estas condiciones:

- su tasa de incidencias supera el 20%
- en algún día tiene menos de 8 operaciones totales

- el total semanal de incidencias supera 15
-

b) CentroSeguridad

Hereda de `CentroOperativo`.

Atributo propio

- `int inspeccionesEspeciales`

Índice de eficiencia

Se calcula así:

$$\text{indice} = \text{calcularTotalOperaciones}() - \text{calcularTotalIncidencias}() * 0.5 + \text{inspeccionesEspeciales} * 2$$

Auditoría

Un centro de seguridad necesita auditoría si se cumple al menos una de estas condiciones:

- existe algún turno con 0 operaciones
 - existe algún turno con más incidencias que operaciones
 - el total semanal de operaciones es menor que 35
-

6) Programa principal

En la clase `Principal`, realiza lo siguiente.

a) Crear un array polimórfico

Crea un array de `CentroOperativo` con 6 posiciones:

- 3 `CentroDistribucion`
- 3 `CentroSeguridad`

Cada objeto debe tener:

- código
- zona
- responsable
- matriz de operaciones

- matriz de incidencias
 - atributo específico de su subclase
-

b) Mostrar toda la información

Para cada centro, mostrar:

- datos generales
 - responsable
 - resumen semanal completo
 - total de operaciones
 - total de incidencias
 - tasa de incidencias
 - índice de eficiencia
 - si necesita auditoría o no
-

c) Centro con mayor índice de eficiencia

Mostrar:

- código
 - tipo real
 - responsable
 - índice
-

d) Centro con peor tasa de incidencias

Mostrar:

- código
 - tasa
 - tipo real
-

e) Contar tipos

Usando `instanceof`, mostrar:

- cuántos centros de distribución hay
 - cuántos centros de seguridad hay
-

f) Calcular medias globales por turno

Teniendo en cuenta todos los centros y todos los días, mostrar:

- media de operaciones del turno de mañana
 - media de operaciones del turno de tarde
 - media de incidencias del turno de mañana
 - media de incidencias del turno de tarde
-

g) Buscar el día más problemático

Se considera “día problemático” el día con mayor número total de incidencias, sumando todos los centros.

Mostrar:

- día de la semana
 - total de incidencias de ese día
-

h) Mostrar el centro más equilibrado

Se considera centro más equilibrado el que tenga **menor diferencia absoluta** entre:

- operaciones totales
- e incidencias totales

Mostrar:

- código
 - diferencia
 - tipo real
-

i) Mostrar solo los centros cuyo responsable tenga antigüedad superior a 10 años

Mostrar:

- código del centro
 - nombre del responsable
 - antigüedad
 - tipo de centro
-

j) Mostrar todos los centros que cumplan simultáneamente:

- más de 40 operaciones totales
- menos de 10 incidencias totales
- no necesitan auditoría